Homework 4: Make Your Agent Hire Another Agent

Joaquin Laria, MAS.665 AI Studio, October 2026. Built automatically from the agent's memory on 13 Oct 2026 02:32 UTC.
Code (optional): forum_agent/hw4.py in https://github.com/JoaquinLaria/job-application-agent

1. Task and service

Offered. A job-posting eligibility screen (service offer, entry 232503), the screening step of the job-application agent I built in Homeworks 1 and 2. Given a posting and a synthetic profile it returns ELIGIBLE, STOP or UNCLEAR, each hard requirement with the exact sentence that states it, and any disqualifier. Before replying, code checks that every quote appears word for word in the posting and drops any that do not.

Delegated. The agent has two tasks, in order of preference. (1) Adversarial test cases for `plain()`, the real function my forum agent uses to turn Canvas HTML into text; a bug there breaks fingerprints, duplicate detection and read-back. (2) An independent check of 8 claims my screen makes about a fictional 10-line job posting, two of them deliberately false.

How it chose. No partner is hard-coded. On each discovery run the agent reads every service offer, and the model scores each one 0 to 10 against the task. Code adds a responsiveness signal (the provider's own replies in its thread, requests waiting over 12 hours) and excludes providers that have answered nobody while three or more requests wait; that removed a test-design offer (231986) that matched on paper but had 11 unanswered requests. A provider is hired only at a score of 6 or more.

Task 1, top offers	Fit (0-10)	Provider replies	Requests waiting >12h
232490	8	not logged	not logged
232431	4	not logged	not logged
232434	3	not logged	not logged
232227	3	not logged	not logged

Task 2, top offers	Fit (0-10)	Provider replies	Requests waiting >12h
232260	10	3	0
232500	4	0	0
232434	3	0	0
232433	2	1	0

2. Technical setup

- **Harness and model.** A Python agent reusing my HW3 agent's Canvas client. Claude Sonnet through the MIT Parley gateway ranks offers, screens postings and, only as a fallback, reads values out of a reply. Acceptance is always decided by code.
- **Canvas.** REST API with my own token from HW3, held in a GitHub Actions secret.
- **When it runs.** GitHub Actions cron, twice an hour from 12 to 14 October (GitHub delays or skips some scheduled runs; a few cycles were started by hand). Each cycle: read the forum, answer requests in my offer thread, then advance the current hire.
- **Memory and replies without relay.** state/hw4_memory.json, committed after every cycle, holds my offer id, handled requests, each hire (task, provider, request id, deadline, status, deliveries and check results), a pending-write record and a failure counter; hw4_runs.jsonl logs every cycle (8 so far). A hire moves from requested to checked, then accepted, one correction, or rejected; a decline or a missed deadline sends the agent to the next suitable provider (at most three, one at a time).

3. Results and verification

Hire 1: test cases for `plain()`. Provider offer 232490, our request 232505, deadline 2026-10-13 21:00 UTC. The provider declined in 232551: its static check refuses code that imports modules, and `plain()` imports `html` and `re`. My agent recorded the decline and looked for another provider. No other offer scored 6 or more for this task, so it moved to its second task.

Hire 2: independent check of the screen's 8 claims. Provider offer 232260, our request 232601, deadline 2026-10-14 01:00 UTC. Waiting for the provider's reply when this PDF was built. The checks below are the ones the agent will run on the reply; the hire stays open until the deadline.

Checks the agent runs. Task 1: read the test values without running any peer code (each value must appear verbatim in the reply), run them against the real function, then against six deliberately broken versions; accept at 8+ agreeing pairs, 80% agreement and 4 of 6 broken versions caught. Task 2: compare each verdict with expected verdicts sealed in

claims_answer_key.json, committed to the repository before the request and quoted by SHA-256 in it; confirm every cited line exists and the deciding line is cited; accept at 7 of 8 including both planted false claims.

4. Requests to my service

Request	Action	Our reply
No other agent had requested the service when this PDF was built.		

5. Safety controls

- **Credentials.** Two Actions secrets, masked in logs, never written or posted. Outgoing text is blocked if it contains links, email addresses or key-shaped strings.
- **Untrusted peers.** Peer text reaches the model only inside a `<peer>` wrapper with `<` escaped, as data. Peer code is never executed: values are read, not run. The model never decides acceptance.
- **Control line and write limit.** Before every write the agent reads the HW4 description and writes only on `COURSE-TEAM CONTROL: RUNNING`. At most one write per cycle and three attempts per rolling hour, counting my HW3 agent's posts.
- **Verified, no duplicates.** Intent is saved before each write and every post is read back by id. The first request's read-back failed because the fingerprint was taken from text that itself contained HTML examples. The post was on Canvas, so the next cycle adopted it as the request instead of resending (I added the request's details to memory by hand once, since the first version did not save them); the fingerprint now uses the HTML actually sent, and the pending record carries the full request.
- **Stops.** Backoff on reads and model calls, never a blind retry of a write, a halt after three failed cycles in a row, and no action outside the run window.